DATA ENGINEERING

Graph Databases Visual Guide

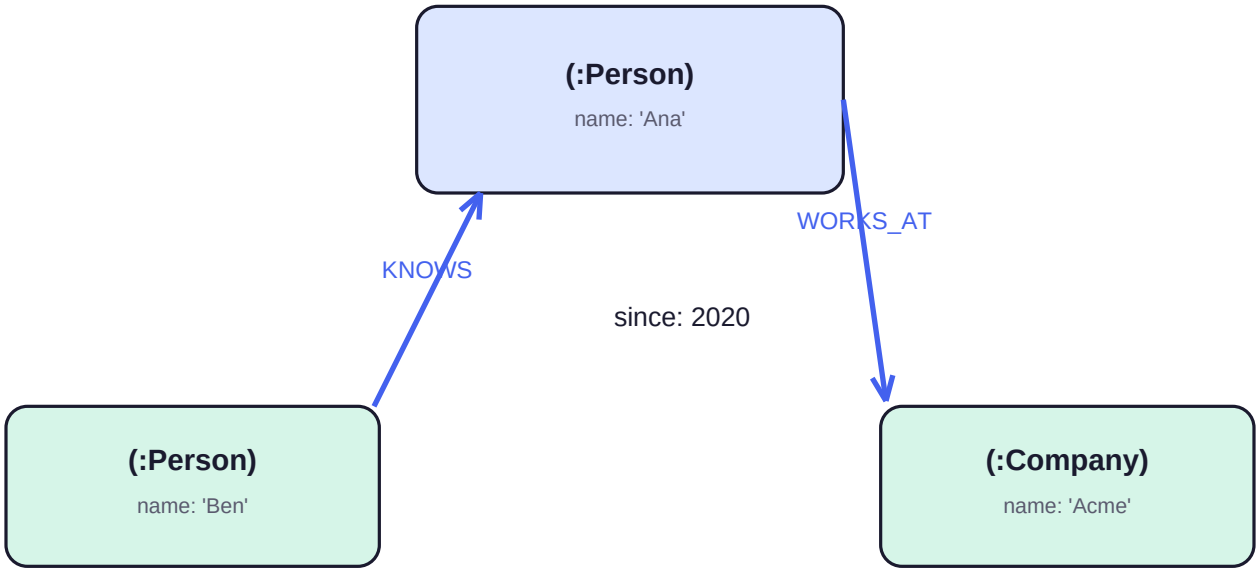
When relationships ARE the data.
Neo4j, traversals, and Cypher.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Property Graph Model

Nodes, relationships, and properties on both.



Relationships are first-class: typed, directed, and carrying their own properties. No join tables — the connection is stored, not computed.

STEP 2/6

Why Graphs Beat Joins Here

Multi-hop queries explode in SQL, stay constant in a graph.

SQL: friends-of-friends-of-friends = 3 self-joins over a 100M-row edge table. Each join multiplies the working set.

Graph: follow pointers from the start node. Cost depends on the neighborhood size, not the total graph size.

SQL multi-hop

O(joins × table size)

Graph traversal

O(visited nodes)

Index-free adjacency: each node physically points to its neighbors. Traversal is pointer-chasing, not index lookups.

STEP 3/6

Cypher Basics

ASCII-art query language: draw the pattern you want.

```
MATCH (a:Person {name:'Ana'})-[:KNOWS]->(friend)
RETURN friend.name;

MATCH (a:Person)-[:KNOWS*2..3]->(fof)
WHERE a.name = 'Ana'
RETURN DISTINCT fof.name;
```

(a)-[:KNOWS]->(b) literally draws the relationship. *2..3 means '2 to 3 hops'. Variable-length paths are one line in Cypher, pages of SQL.

- CREATE builds nodes and relationships in one statement
- MERGE = get-or-create (idempotent writes)
- PROFILE shows the traversal plan — watch for cartesian products

STEP 4/6

Traversal Patterns

The queries graphs were born for.

Path finding

Shortest path between two nodes — fraud rings, routing

Neighborhood

Everything within N hops — recommendations, blast radius

Pattern matching

Find subgraphs shaped like X — money-laundering motifs

Centrality

PageRank, betweenness — who matters in the network

Graph Data Science library adds 65+ algorithms (community detection, embeddings) runnable directly on the stored graph.

STEP 5/6

When Graphs Win (and Don't)

Reach for a graph when the question is about connections.

GRAPHS WIN

- Fraud / AML networks
- Recommendations
- Knowledge graph + LLM

SKIP GRAPHS

- Simple CRUD with few joins
- Heavy aggregations over facts
- Tabular reporting

Graphs are bad at what warehouses are good at: scanning millions of rows and summing them. Use both — graph for connections, warehouse for aggregates.

STEP 6/6

Graph Modeling Tips

Model for the questions you'll ask.

- Relationships over properties: 'WORKS_AT' beats employer_name string
- Keep graphs shallow: 2-4 hop queries are fast; 10-hop scans everything
- Index the starting points — traversals begin with a lookup
- Time-version relationships (since/until) instead of deleting history
- Supernodes (a node with millions of edges) kill traversals — split them

The #1 graph performance bug: starting a traversal without an indexed anchor node. Always MATCH the start node by indexed property first.

STEP ✓I✓

Key Takeaways

Graph Databases — Remember

- Property graphs store nodes, typed relationships, and properties — connections are stored, not joined.
- Index-free adjacency makes multi-hop traversals constant in neighborhood size, not table size.
- Cypher draws patterns as ASCII art; variable-length paths are one line.
- Reach for graphs for fraud, recommendations, and knowledge — not for aggregations or CRUD.
- Model relationships explicitly, keep traversals shallow, and always start from an indexed node.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev